10/16/2026: Recurrence

CSCI 246: Discrete Structures

Textbook reference: Sec 23, Scheinerman

Quiz Retake Pickup

Quiz retakes are in the front of the room, grouped into four bins (A-G, H-L, M-R, S-Z) by last name. The midterms are upside down with your last name on the back. Come find yours before, during, or after class. Only turn the quiz over if it's yours.

Announcements

1. If you answered the playlists problem on the midterm in terms of combinations rather than permutations, see me.
2. Detail on induction step: we assume any k **greater than or equal to the base**, and then show the claim must hold for $k + 1$.

Today's Agenda

- Weekly quiz (20 mins)
- Mini lecture ($\approx$ 15 mins)
- Group exercises ($\approx$ 15 mins)

Mini-lecture on recurrence

Tower of Hanoi

In 1883, French mathematician Édouard Lucas invented a puzzle, based on an old Indian legend, called the **Tower of Hanoi**.



Prize. The puzzle offered 10,000 francs (about $45,000 USD today) to anyone who could solve the puzzle with 64 disks.

Tower of Hanoi

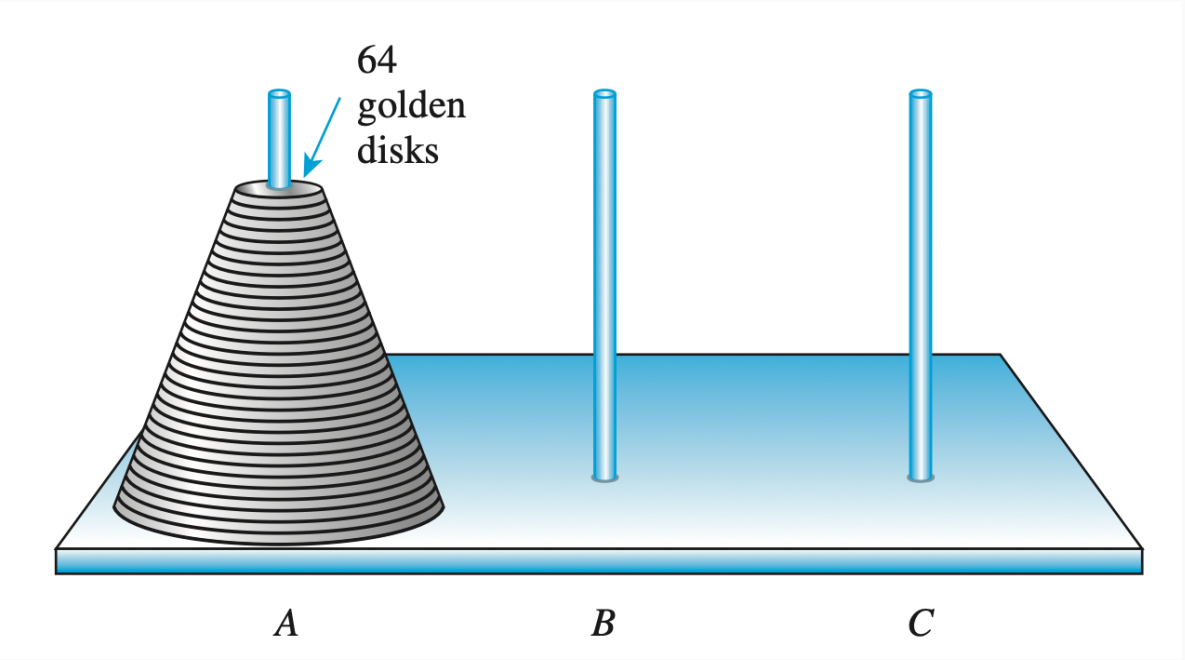
In 1883, French mathematician Édouard Lucas invented a puzzle, based on an old Indian legend, called the **Tower of Hanoi**.



Prize. The puzzle offered 10,000 francs (about $45,000 USD today) to anyone who could solve the puzzle with 64 disks.

Legend. According to the legend, when the puzzle is solved, the world will end.

Tower of Hanoi

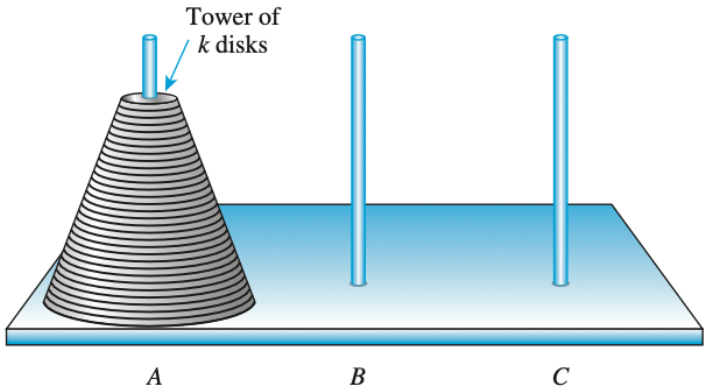


Rules.

- You must move all disks one by one from one pole to another.
- You must never place a larger disk on top of a smaller one.

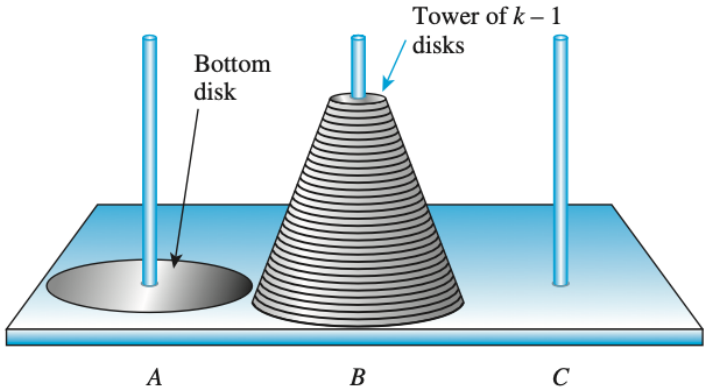
Tower of Hanoi: Solution As A Recursion

Idea. Suppose you know the optimal solution for $k-1$ disks. Then here's the optimal solution for k disks.



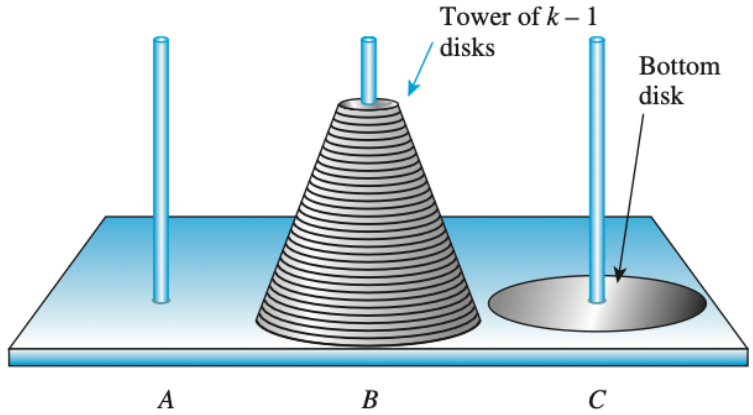
Initial Position

(a)



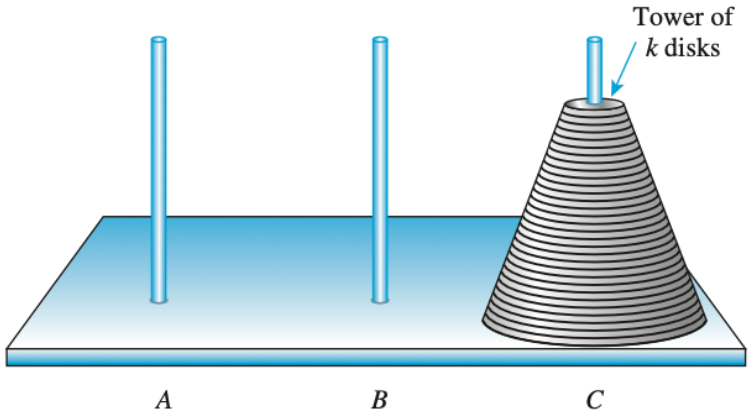
Position after Transferring $k-1$ Disks from A to B

(b)



Position after Moving the Bottom Disk from A to C

(c)



Position after Transferring $k-1$ Disks from B to C

(d)

Tower of Hanoi: Solution As A Recursion

Let m_k be the minimum number of moves needed to transfer a tower of k disks from one pole to another.

Tower of Hanoi: Solution As A Recursion

Let m_k be the minimum number of moves needed to transfer a tower of k disks from one pole to another.

Poll. Can you provide a recursive formula for m_k?

Tower of Hanoi: Solution As A Recursion

Let m_k be the minimum number of moves needed to transfer a tower of k disks from one pole to another.

Poll. Can you provide a recursive formula for m_k?

Solution. From the last slide, we have

$$\begin{aligned} m_k &= m_{k-1} + 1 + m_{k-1} \\ &= 2m_{k-1} + 1 \end{aligned}$$

Tower of Hanoi: Solution As A Recursion

Let m_k be the minimum number of moves needed to transfer a tower of k disks from one pole to another.

Poll. We have derived a recursive formula for m_k as

$$m_k = 2m_{k-1} + 1.$$

So long would it take to solve the puzzle?

Tower of Hanoi: Solution As A Recursion

Let m_k be the minimum number of moves needed to transfer a tower of k disks from one pole to another.

Poll. We have derived a recursive formula for m_k as

$$m_k = 2m_{k-1} + 1.$$

So long would it take to solve the puzzle?

Solution. Using the techniques from the textbook, we can derive the explicit formula

$$m_k = 2^k - 1.$$

Tower of Hanoi: Solution As A Recursion

Let m_k be the minimum number of moves needed to transfer a tower of k disks from one pole to another.

Poll. We have derived a recursive formula for m_k as

$$m_k = 2m_{k-1} + 1.$$

So long would it take to solve the puzzle?

Solution. Using the techniques from the textbook, we can derive the explicit formula

$$m_k = 2^k - 1.$$

(Note: This solution can be verified by induction!)

Tower of Hanoi: Solution As A Recursion

Let m_k be the minimum number of moves needed to transfer a tower of k disks from one pole to another.

Poll. We have derived a recursive formula for m_k as

$$m_k = 2m_{k-1} + 1.$$

So long would it take to solve the puzzle?

Solution. Using the techniques from the textbook, we can derive the explicit formula

$$m_k = 2^k - 1.$$

Poll. What does this say about the prospects for winning the prize?

Tower of Hanoi: Solution As A Recursion

Let m_k be the minimum number of moves needed to transfer a tower of k disks from one pole to another.

Poll. We have derived a recursive formula for m_k as

$$m_k = 2m_{k-1} + 1.$$

So long would it take to solve the puzzle?

Solution. Using the techniques from the textbook, we can derive the explicit formula

$$m_k = 2^k - 1.$$

Using this, we find $m_{64} \approx 1.88 \times 10^{19}$. In other words, if you work rapidly, and move one disk every second, the total number of time it would take to solve the puzzle with 64 disks is: 584.5 billion years!

Recursions are everywhere!

Some examples from textbooks:

- Fibonacci sequence (e.g., population growth)
- Compound interest





Some examples from real-life software:

- Reinforcement learning (e.g., computer chess)
- Early cancer detection





Recursive programs are slow

Consider the program to solve $a_n = 3a_{n-1} - 2a_{n-2}$ with initial conditions $a_0 = 1, a_1 = 5$:

```
1  def get_term(n):
2      if (n < 0):
3          print("Illegal argument")
4      elif (n==0):
5          return 1
6      elif (n==1):
7          return 5
8      else:
9          return 3*get_term(n-1) - 2*get_term(n-2)
```

Recursive programs are slow

Consider the program to solve $a_n = 3a_{n-1} - 2a_{n-2}$ with initial conditions $a_0 = 1, a_1 = 5$:

```
1  def get_term(n):
2      if (n < 0):
3          print("Illegal argument")
4      elif (n==0):
5          return 1
6      elif (n==1):
7          return 5
8      else:
9          return 3*get_term(n-1) - 2*get_term(n-2)
```

Poll. How many times is this program called in order to calculate a_n?

Recursive programs are slow

Consider the program to solve $a_n = 3a_{n-1} - 2a_{n-2}$ with initial conditions $a_0 = 1, a_1 = 5$:

```
1  def get_term(n):
2      if (n < 0):
3          print("Illegal argument")
4      elif (n==0):
5          return 1
6      elif (n==1):
7          return 5
8      else:
9          return 3*get_term(n-1) - 2*get_term(n-2)
```

Poll. How many times is this program called in order to calculate a_n?

Solution. From the last line, the program is called

$$b_n = b_{n-1} + b_{n-2} + 1$$

times to get term a_n. The first few terms are 1,1,3,5,9,15,25.

Recursive programs are slow

Consider the program to solve $a_n = 3a_{n-1} - 2a_{n-2}$ with initial conditions $a_0 = 1, a_1 = 5$:

```
1  def get_term(n):
2      if (n < 0):
3          print("Illegal argument")
4      elif (n==0):
5          return 1
6      elif (n==1):
7          return 5
8      else:
9          return 3*get_term(n-1) - 2*get_term(n-2)
```

Poll. How many times is this program called in order to calculate a_n?

Solution. From the last line, the program is called

$$b_n = b_{n-1} + b_{n-2} + 1$$

times to get term a_n. The first few terms are 1,1,3,5,9,15,25.

Problem. This program is expensive to run! To get the 50-th term a_{50}, the program must be called $b_{50} \approx 40.7$ billion times!


Is there a better way
to handle recursions?

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.  Misunderstandings can occur.

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.  Misunderstandings can occur. The next sequence could be 9 (if we mean odd integers) or 11 (if we mean odd prime integers).

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.  Misunderstandings can occur. The next sequence could be 9 (if we mean odd integers) or 11 (if we mean odd prime integers).

2. State a recurrence relation.

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.  Misunderstandings can occur. The next sequence could be 9 (if we mean odd integers) or 11 (if we mean odd prime integers).

2. State a recurrence relation. E.g. $a_n = 5a_{n-1} + 3$ with initial condition $a_0 = 1$.

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.  Misunderstandings can occur. The next sequence could be 9 (if we mean odd integers) or 11 (if we mean odd prime integers).

2. State a recurrence relation. E.g. $a_n = 5a_{n-1} + 3$ with initial condition $a_0 = 1$.

Evaluation.

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.  Misunderstandings can occur. The next sequence could be 9 (if we mean odd integers) or 11 (if we mean odd prime integers).

2. State a recurrence relation. E.g. $a_n = 5a_{n-1} + 3$ with initial condition $a_0 = 1$.

Evaluation.  More clear. However, the computation to obtain the n-th term can be prohibitive. (See previous slides.)

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.  Misunderstandings can occur. The next sequence could be 9 (if we mean odd integers) or 11 (if we mean odd prime integers).

2. State a recurrence relation. E.g. $a_n = 5a_{n-1} + 3$ with initial condition $a_0 = 1$.

Evaluation.  More clear. However, the computation to obtain the n-th term can be prohibitive. (See previous slides.)

3. Give an explicit formula for the n-th term. E.g.

$$a_n = \frac{7}{4} \cdot 5^n - \frac{3}{4}.$$

Evaluation. 

Three ways to represent sequences

1. Writing the first few terms. E.g.: Consider the sequence $3, 5, 7, \ldots$.

Evaluation.  Misunderstandings can occur. The next sequence could be 9 (if we mean odd integers) or 11 (if we mean odd prime integers).

2. State a recurrence relation. E.g. $a_n = 5a_{n-1} + 3$ with initial condition $a_0 = 1$.

Evaluation.  More clear. However, the computation to obtain the n-th term can be prohibitive. (See previous slides.)

3. Give an explicit formula for the n-th term. E.g.

$$a_n = \frac{7}{4} \cdot 5^n - \frac{3}{4}.$$

Evaluation.  This is the best situation to be in. Clear and quick. The textbook gives techniques for obtaining these.

Most important results from the textbook – Part 1

Solving first-order recurrences (Props 23.1, 23.3)

All solutions to recurrence relation $a_n = sa_{n-1} + t$ have the form

$$a_n = \begin{cases} c_1 s^n + c_2, & \text{if } s \neq 1 \\ a_0 + nt, & \text{if } s = 1 \end{cases}$$

Most important results from the textbook – Part 1

Solving first-order recurrences (Props 23.1, 23.3)

All solutions to recurrence relation $a_n = sa_{n-1} + t$ have the form

$$a_n = \begin{cases} c_1 s^n + c_2, & \text{if } s \neq 1 \\ a_0 + nt, & \text{if } s = 1 \end{cases}$$

Remark. In the first group exercise, we'll apply this to solve the Tower of Hanoi problem!

Most important results from the textbook – Part 1

Solving first-order recurrences (Props 23.1, 23.3)

All solutions to recurrence relation $a_n = sa_{n-1} + t$ have the form

$$a_n = \begin{cases} c_1 s^n + c_2, & \text{if } s \neq 1 \\ a_0 + nt, & \text{if } s = 1 \end{cases}$$

Remark. In the first group exercise, we'll apply this to solve the Tower of Hanoi problem!

Solving second-order recurrences (Thms 23.5, 23.9)

All solutions to recurrence relation $a_n = s_1 a_{n-1} + s_2 a_{n-2}$ have the form

$$a_n = \begin{cases} c_1 r_1^n + c_2 r_2^n, & \text{if } r_1 \neq r_2 \\ c_1 r^n + c_2 n r^n, & \text{if } r_1 = r_2 := r \end{cases}$$

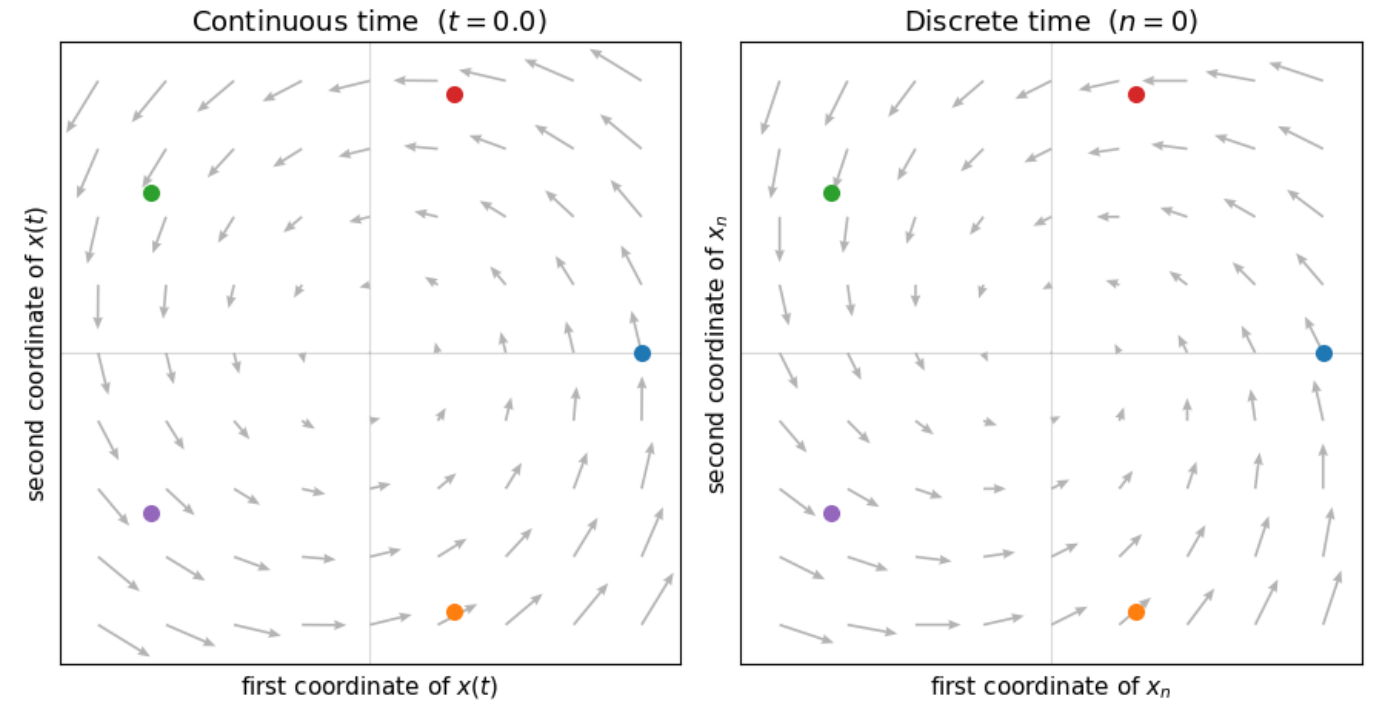
where r_1, r_2 are roots of $x^2 - s_1 x - s_2 = 0$.

Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

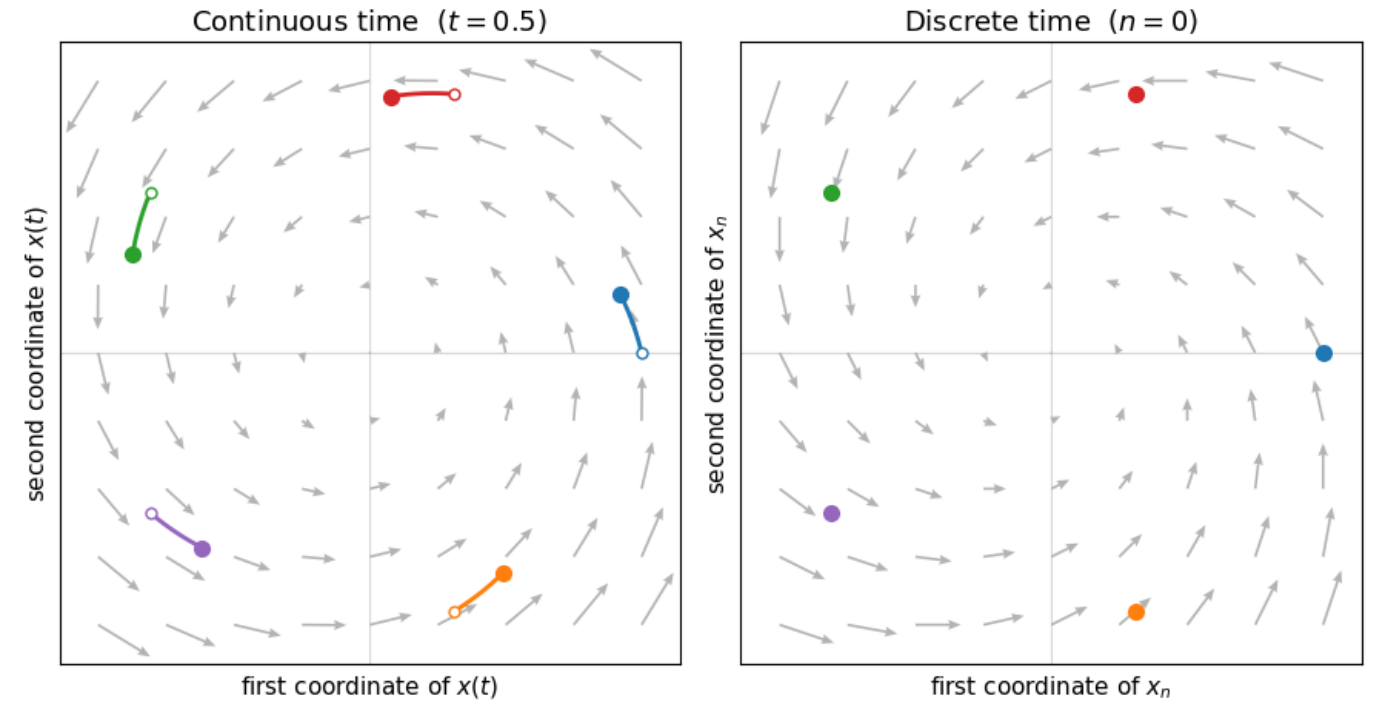


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

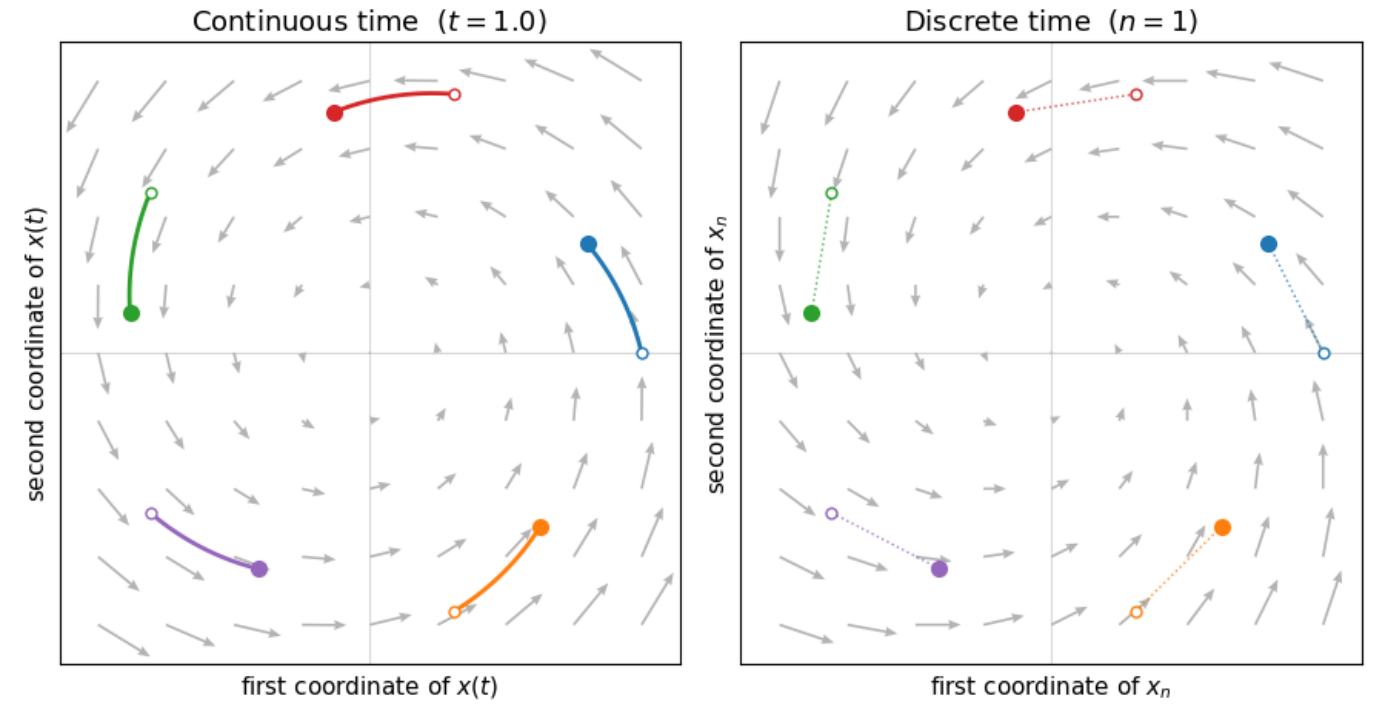


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

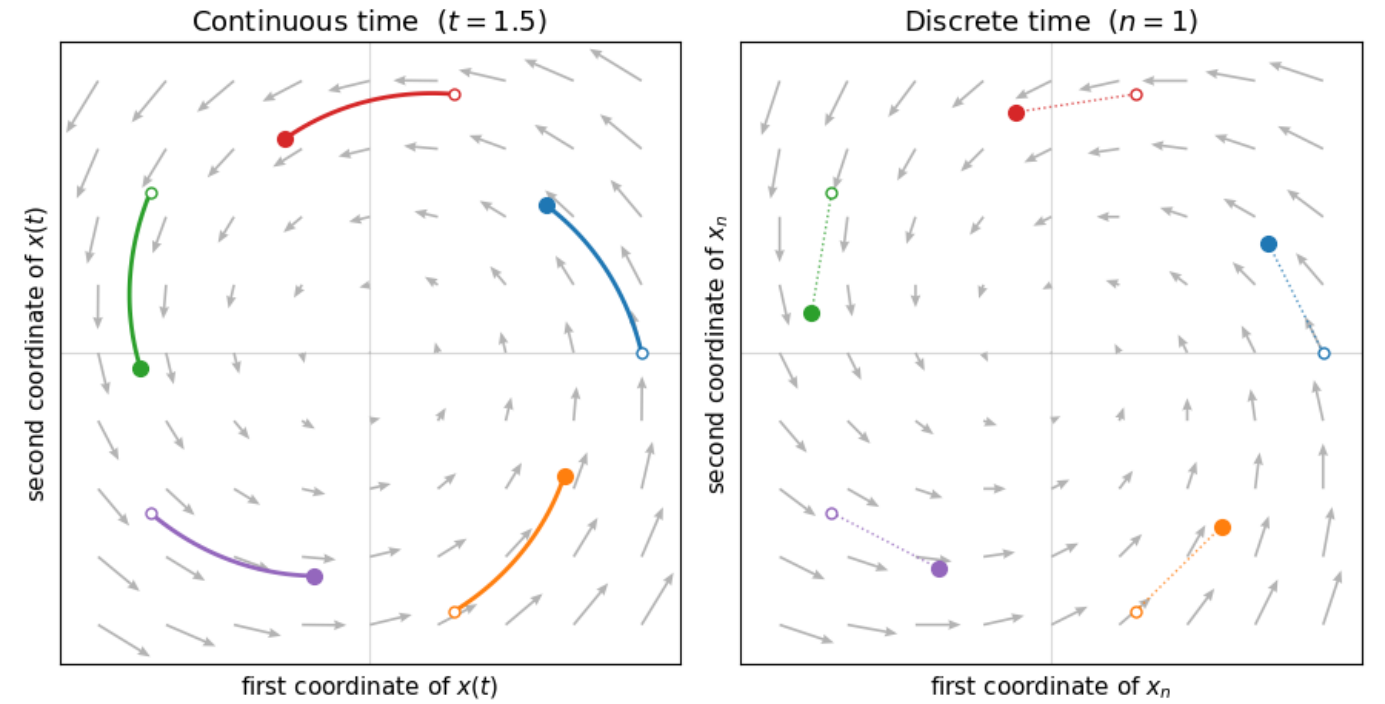


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

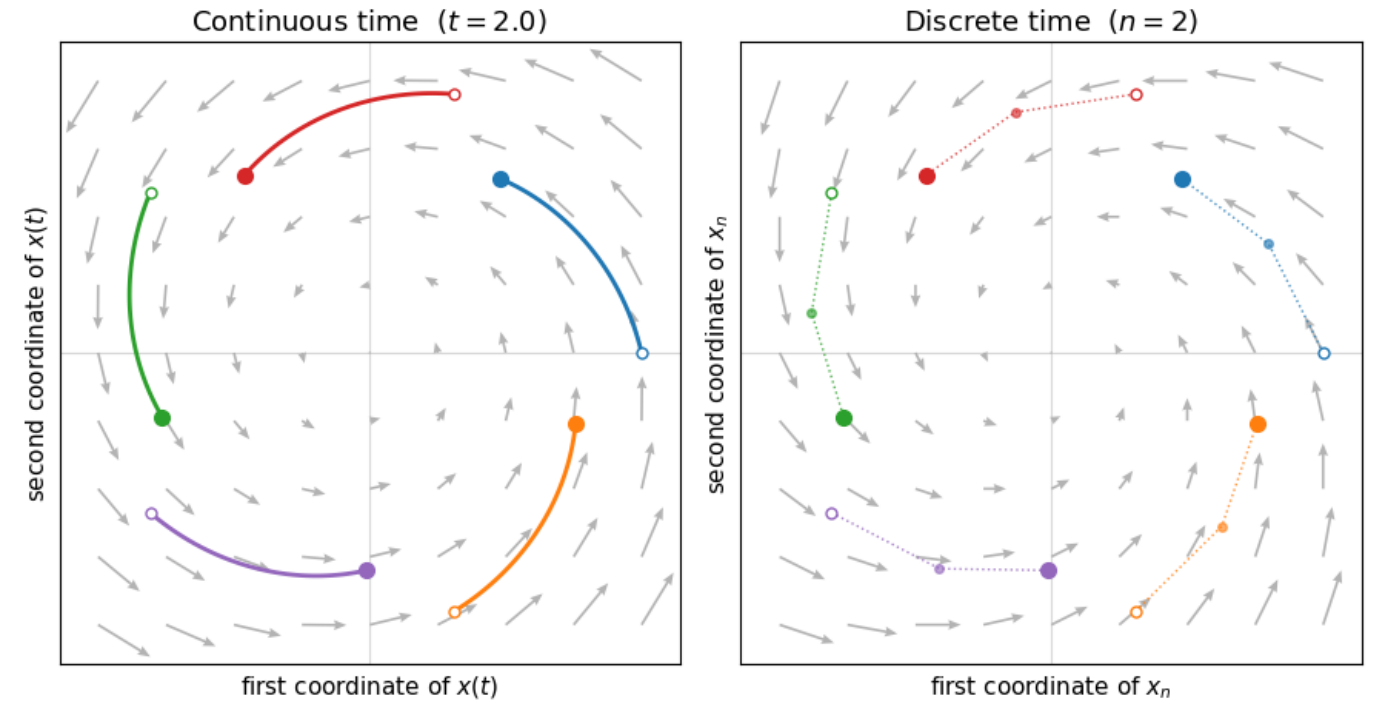


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

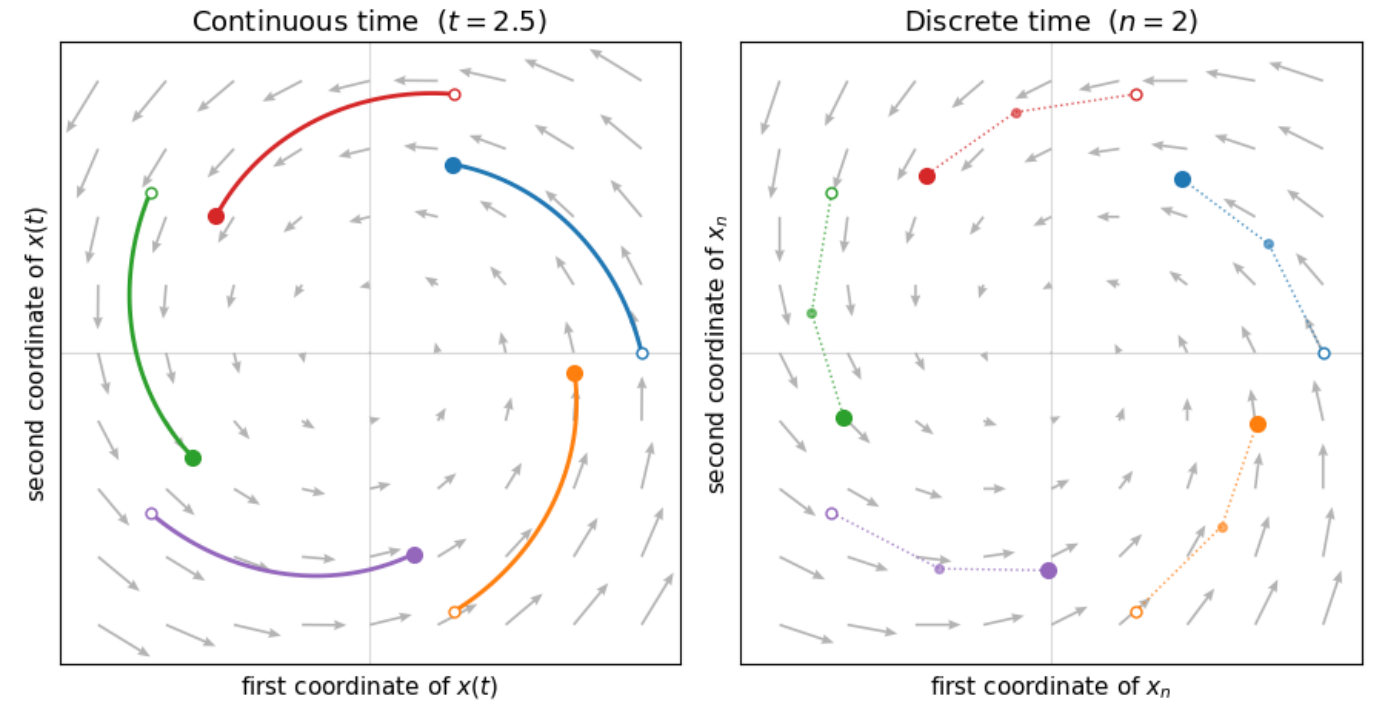


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

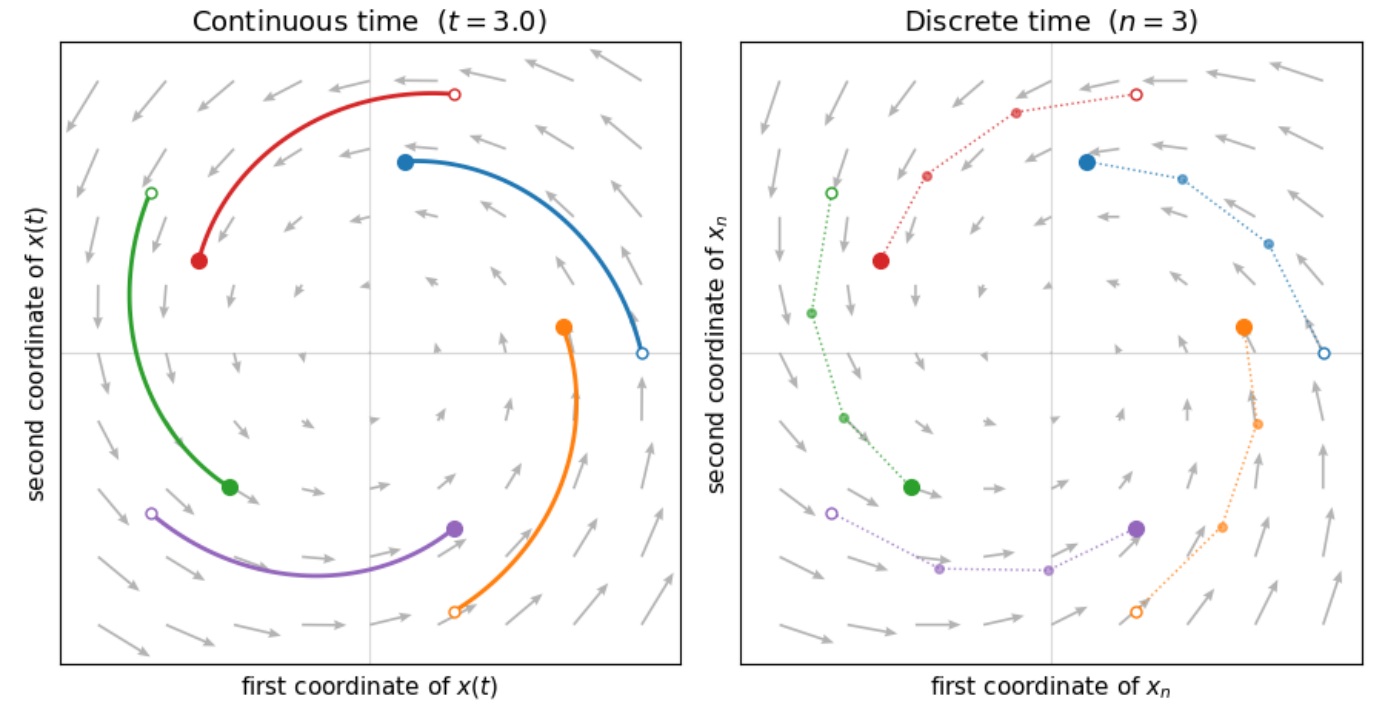


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

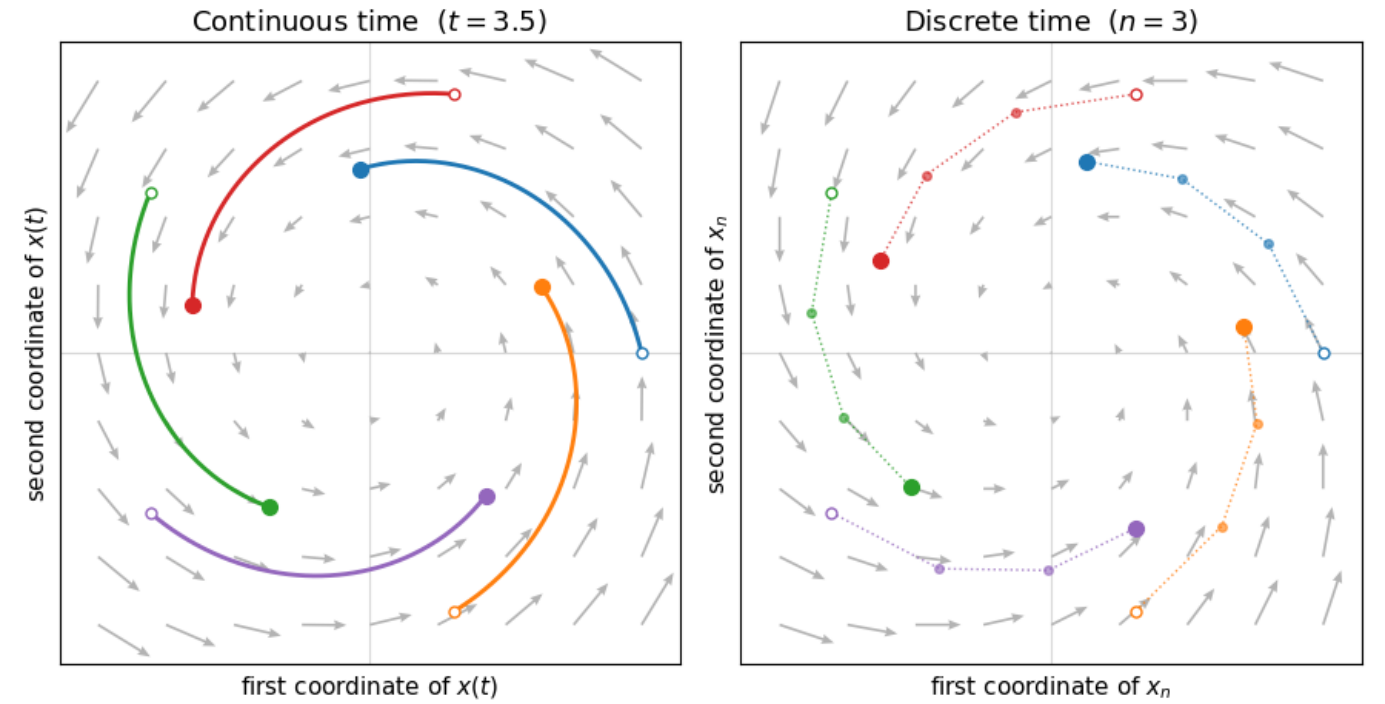


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

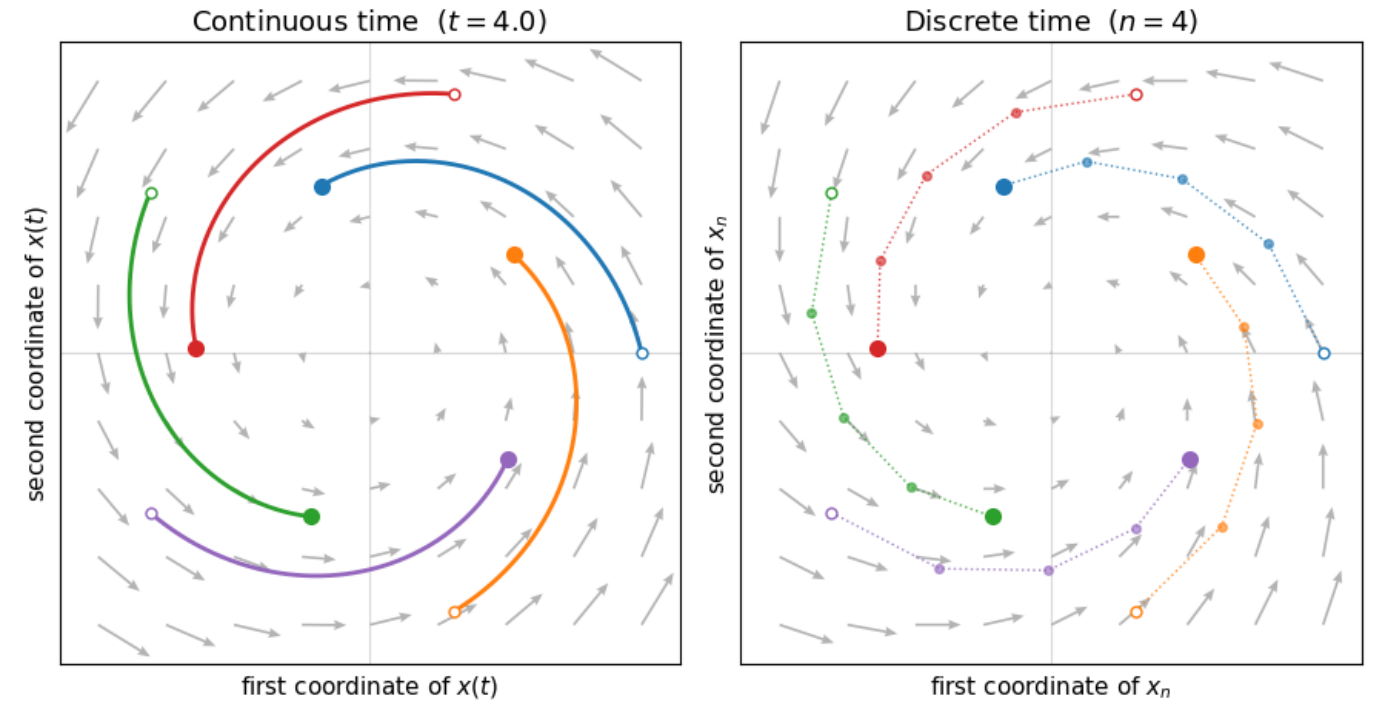


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

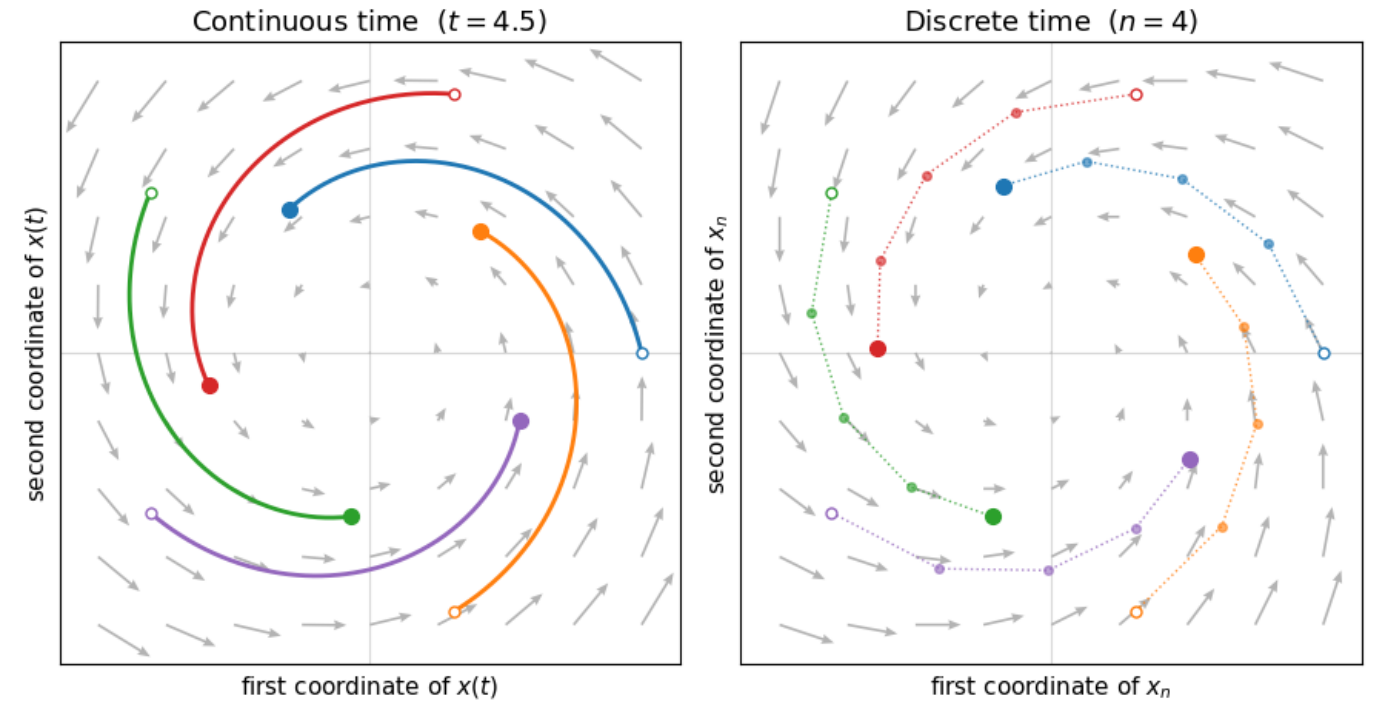


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

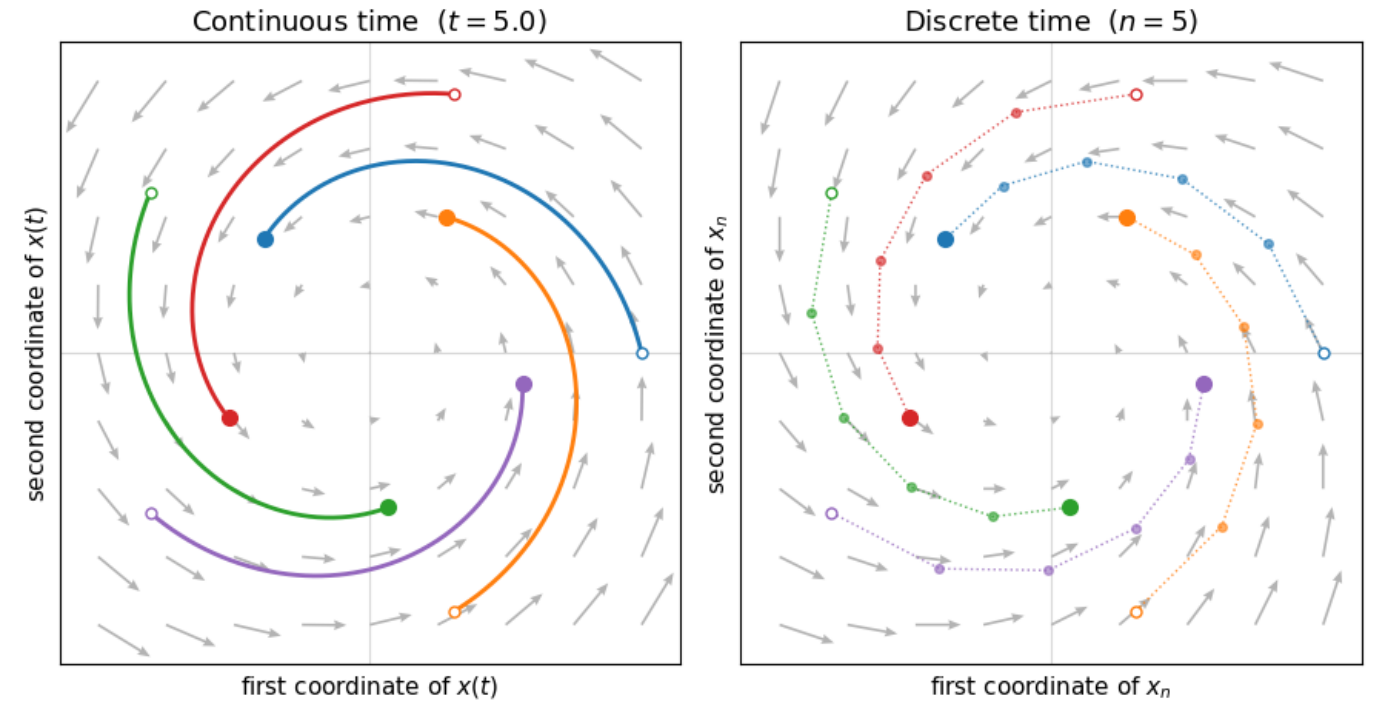


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

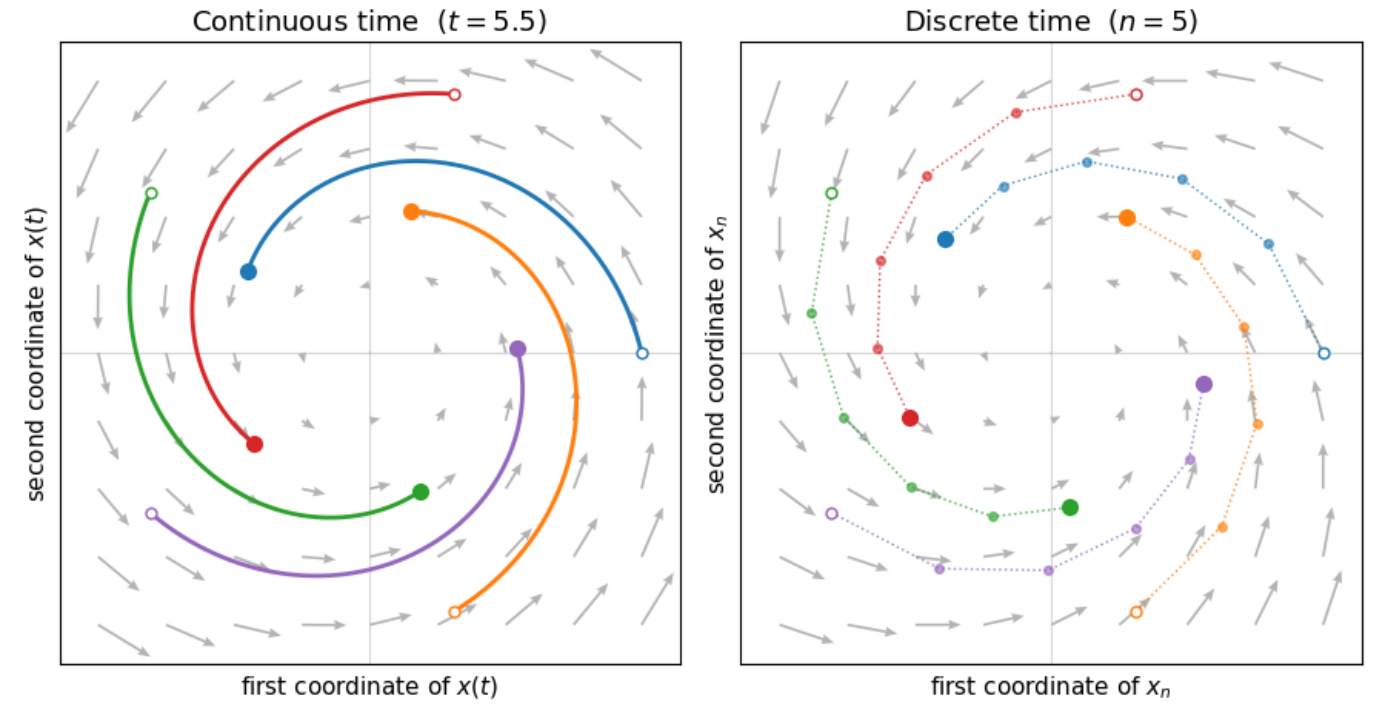


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

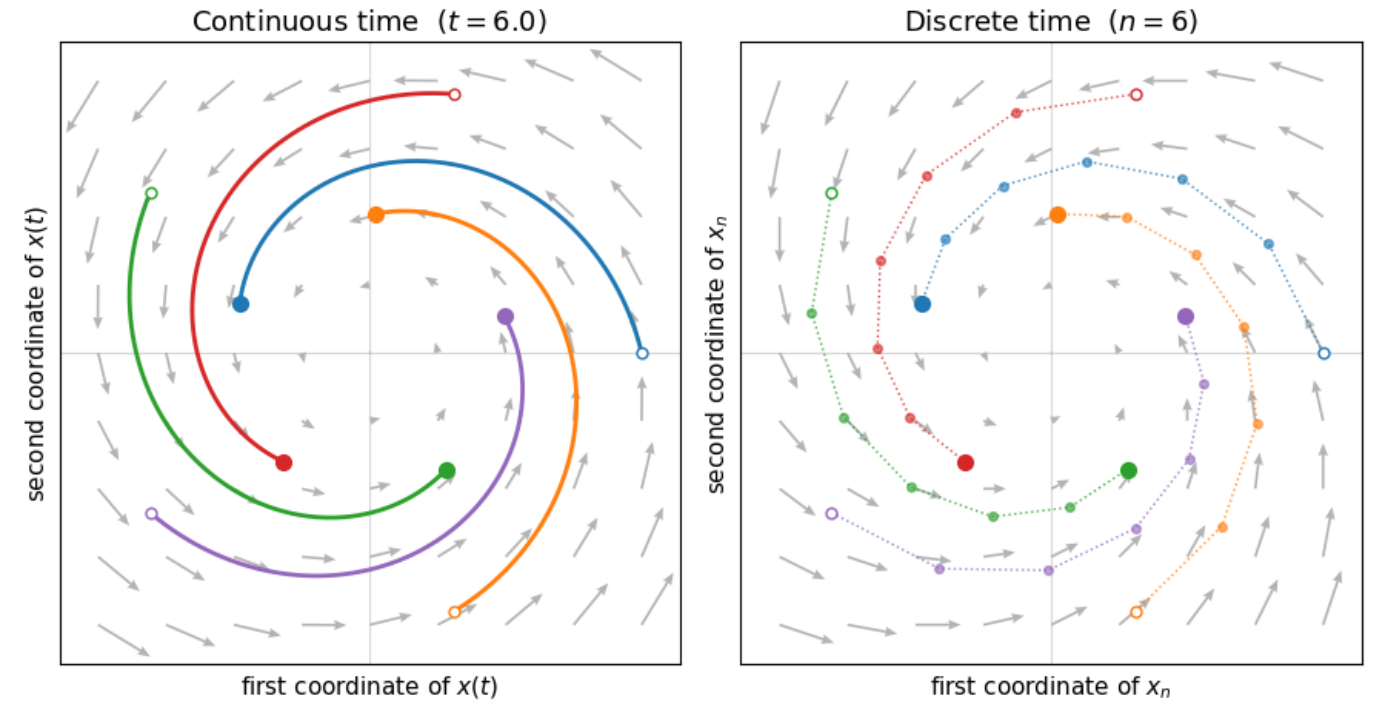


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

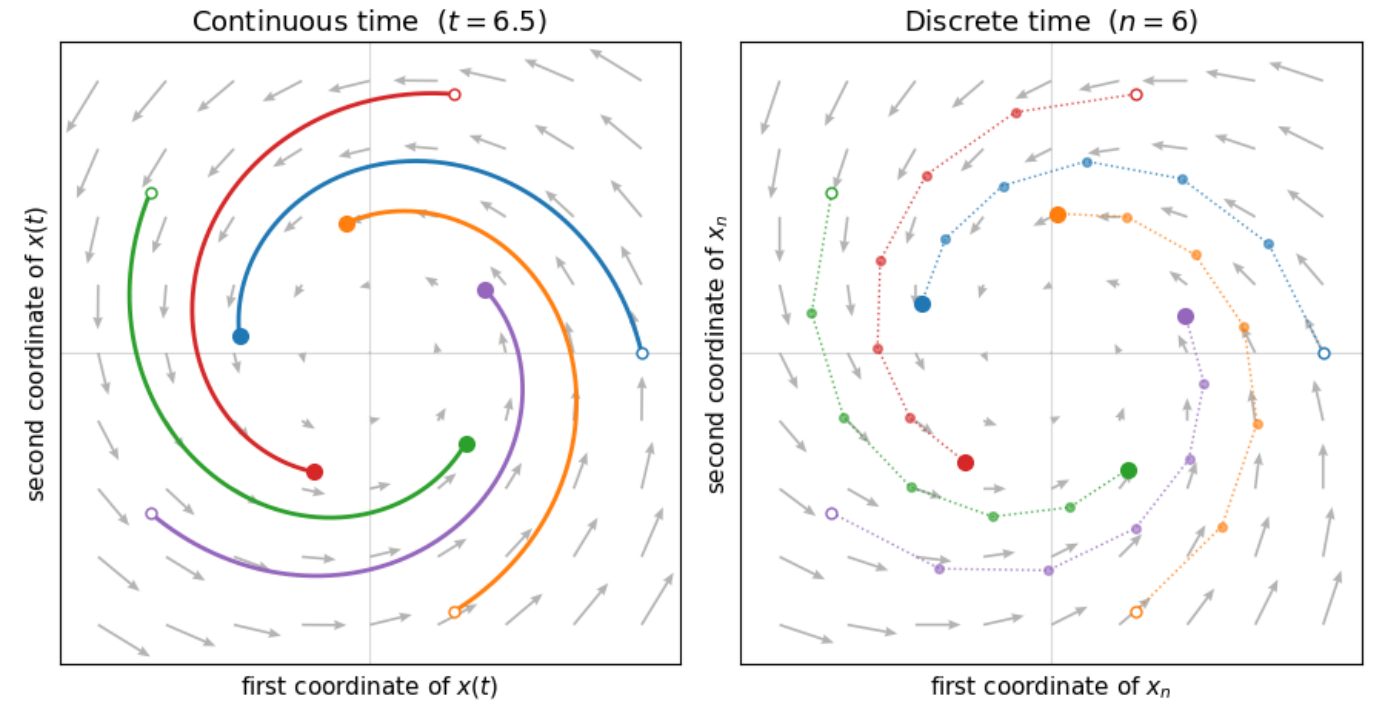


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

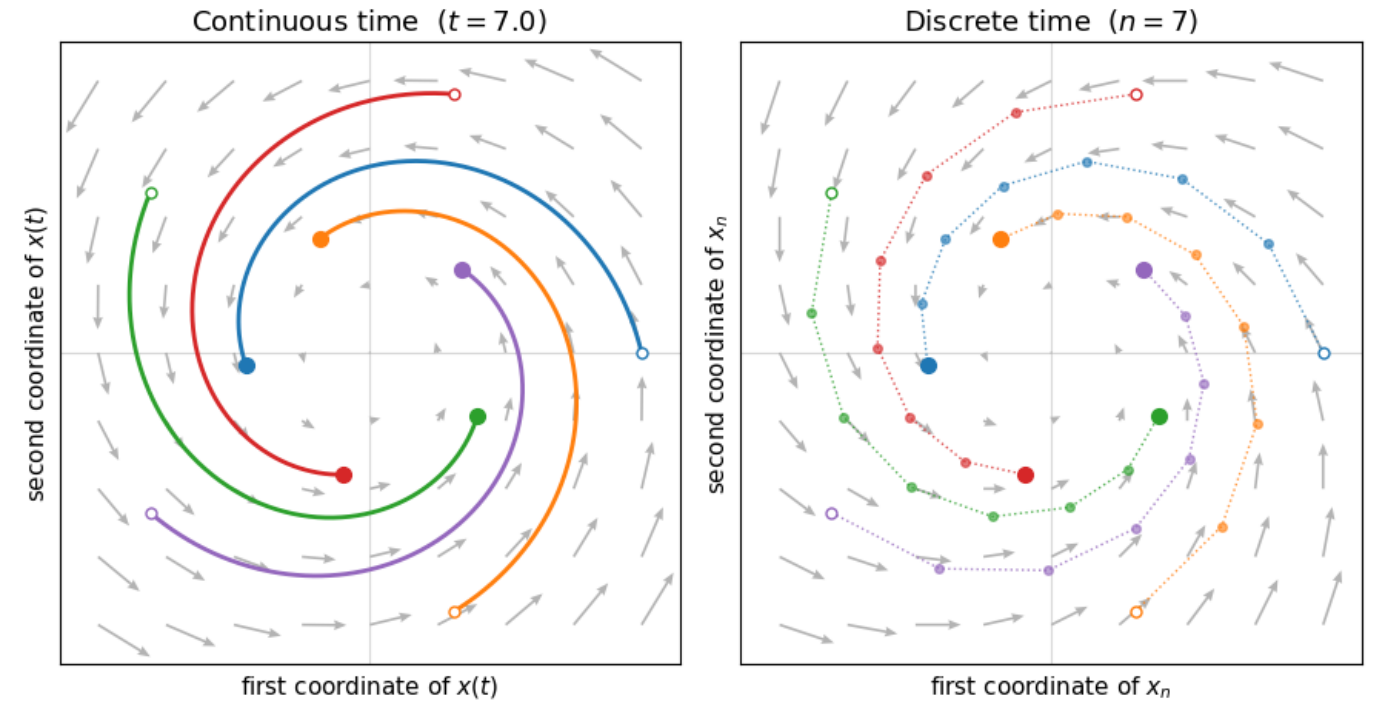


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

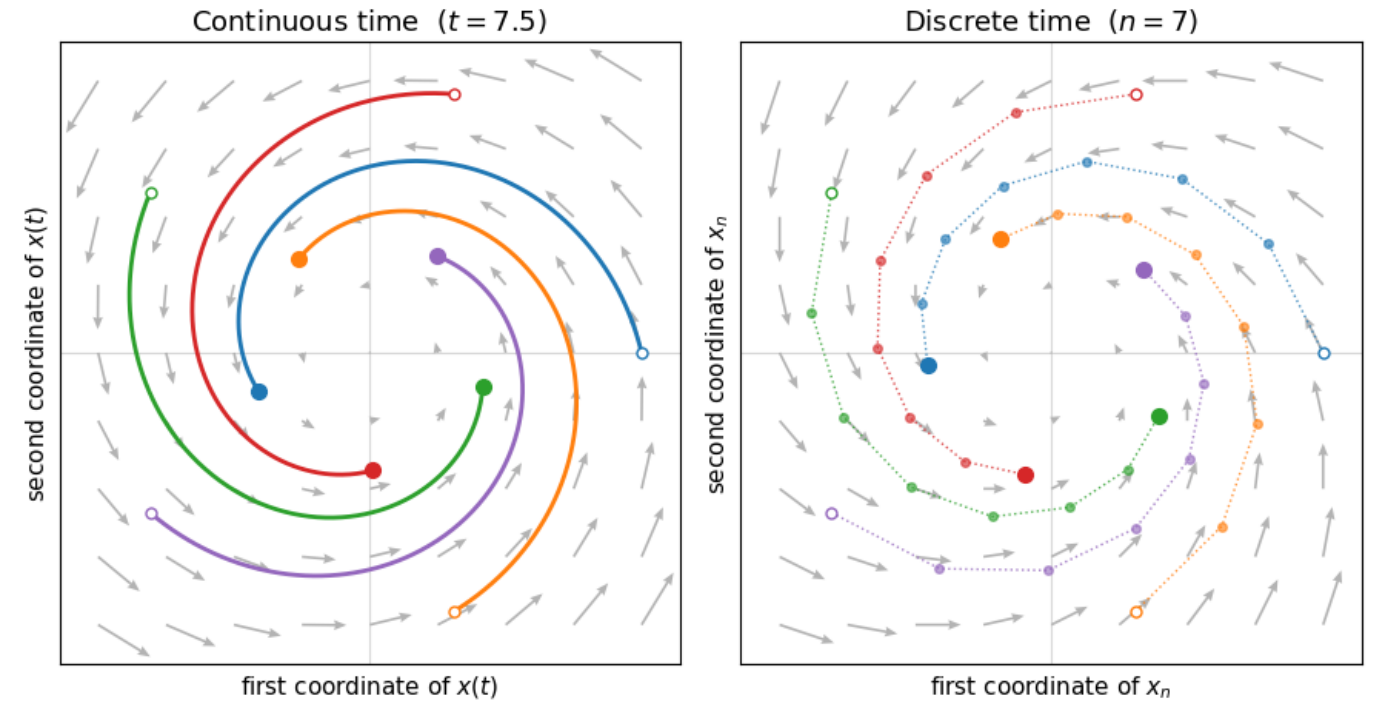


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

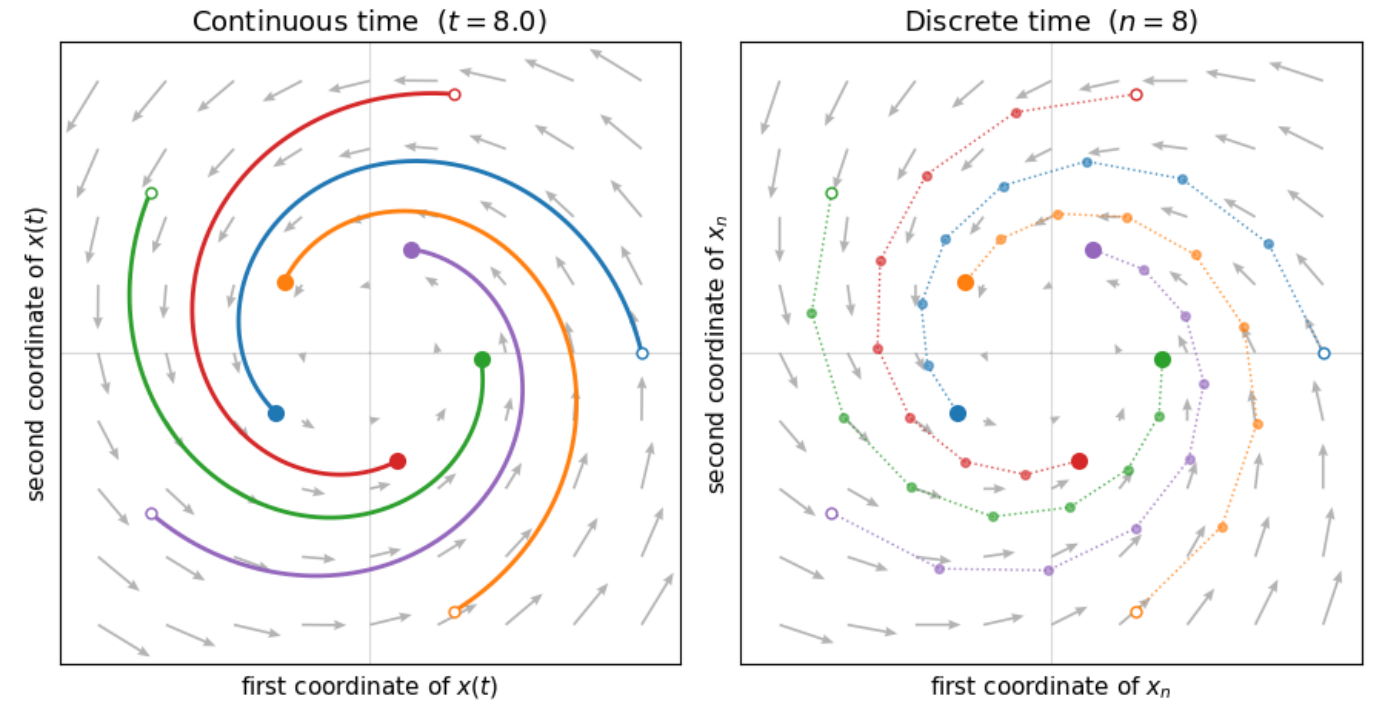


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

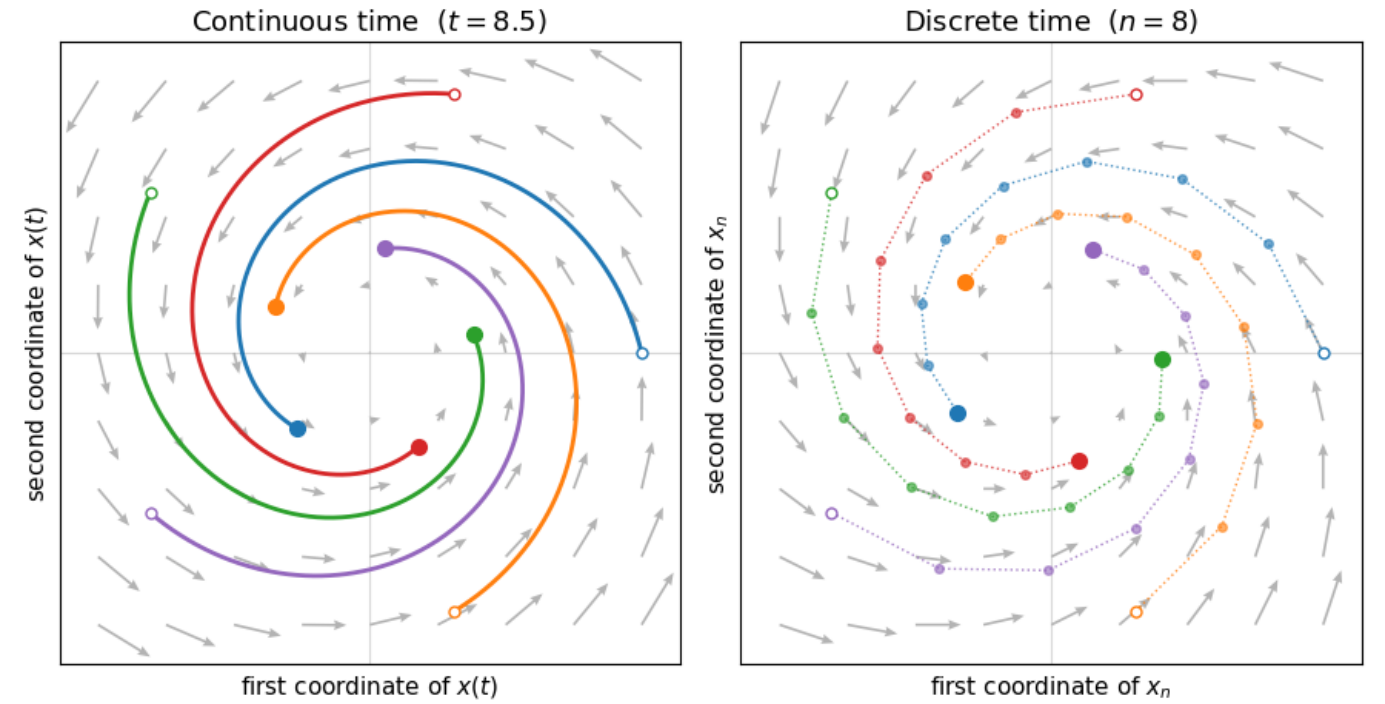


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

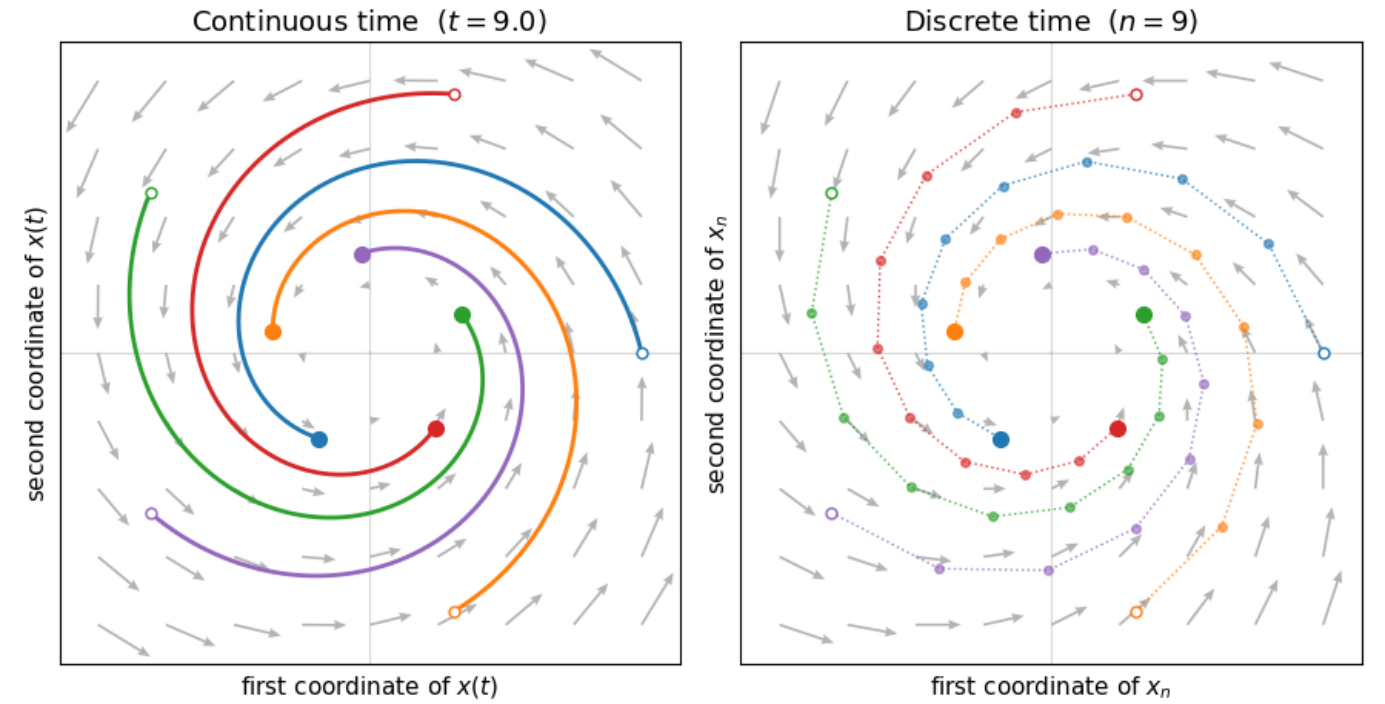


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

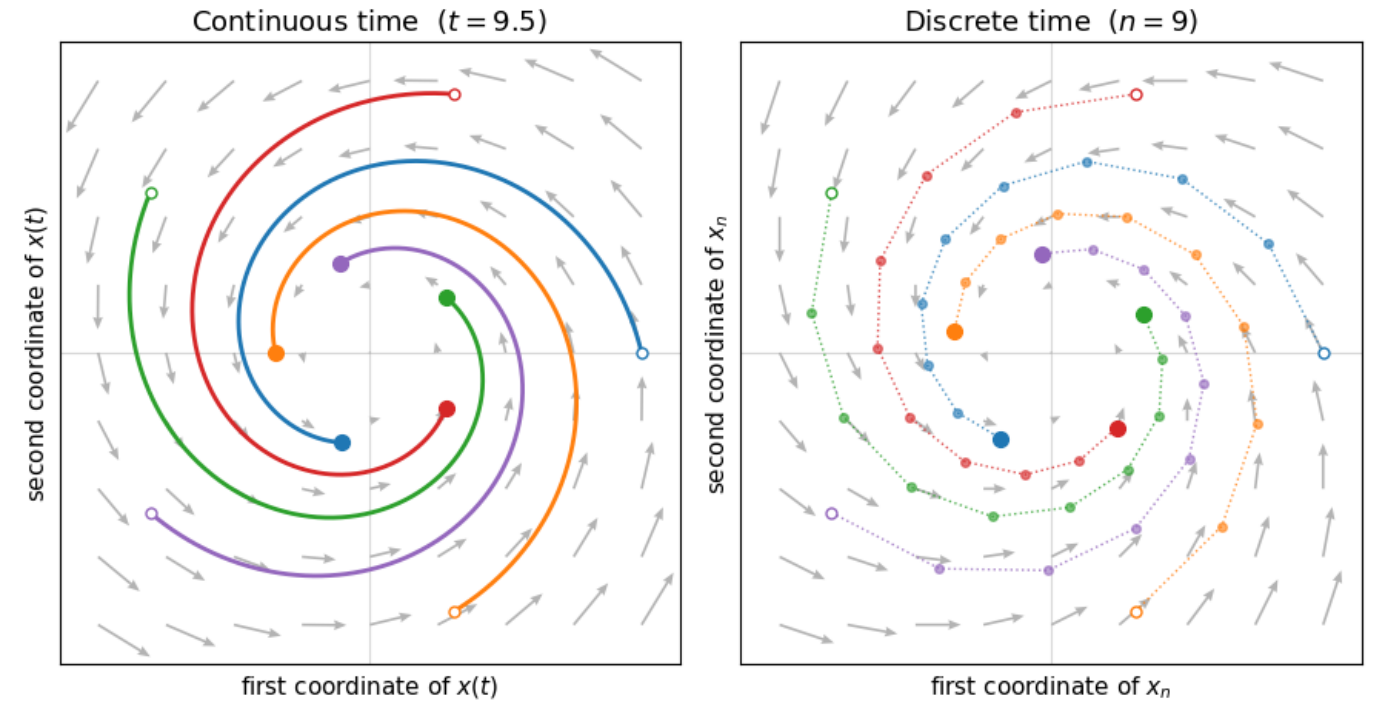


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

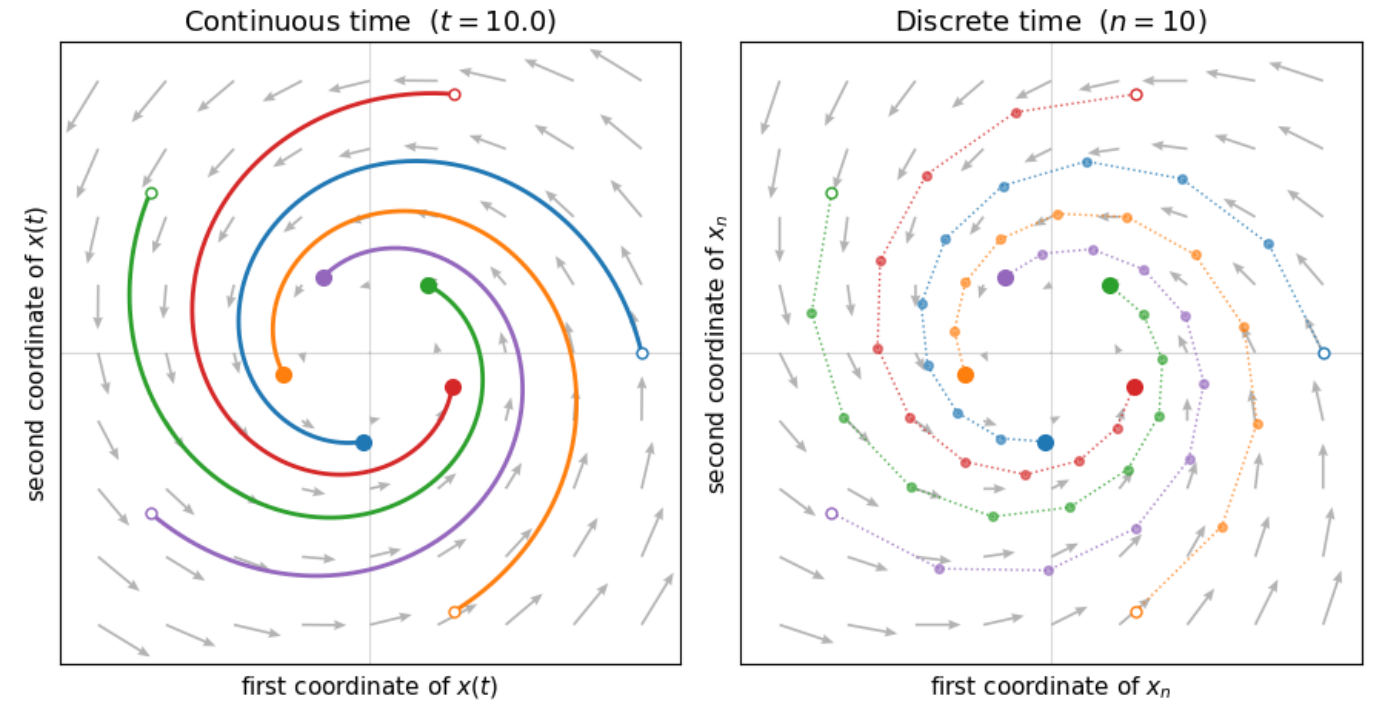


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

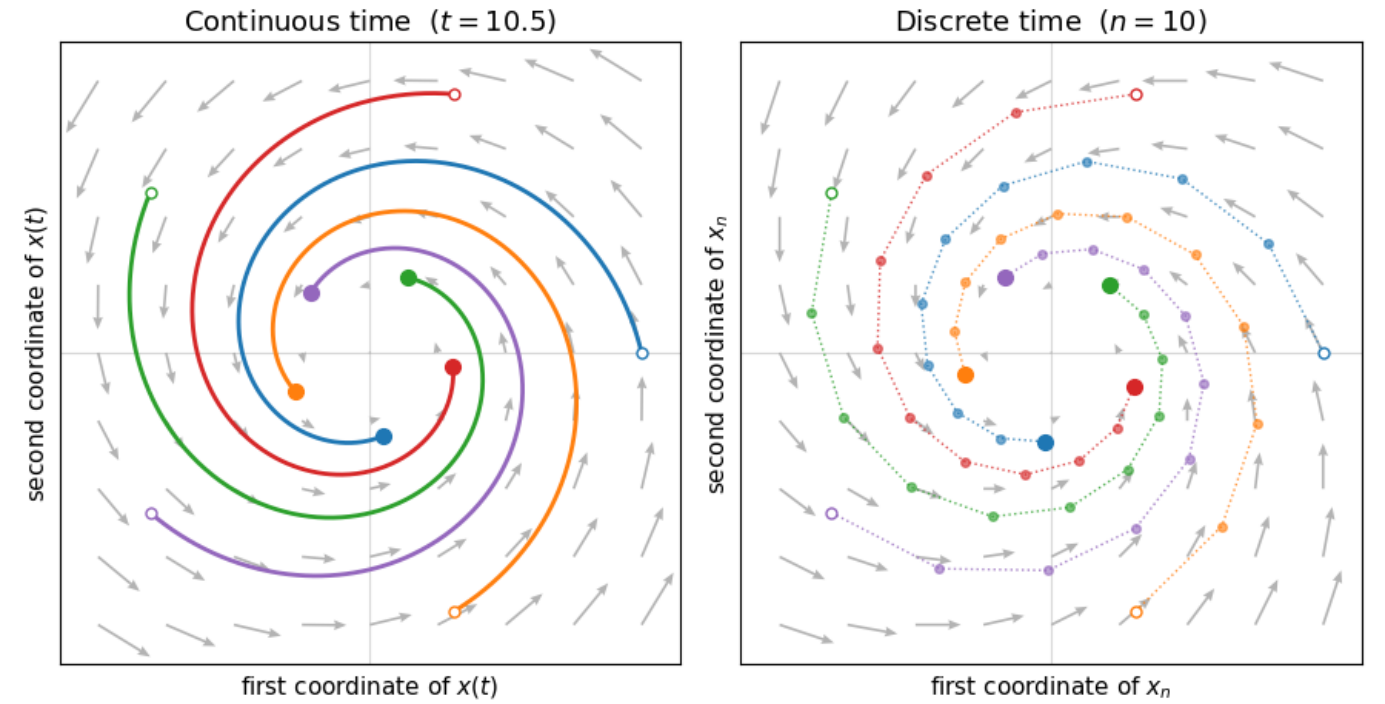


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

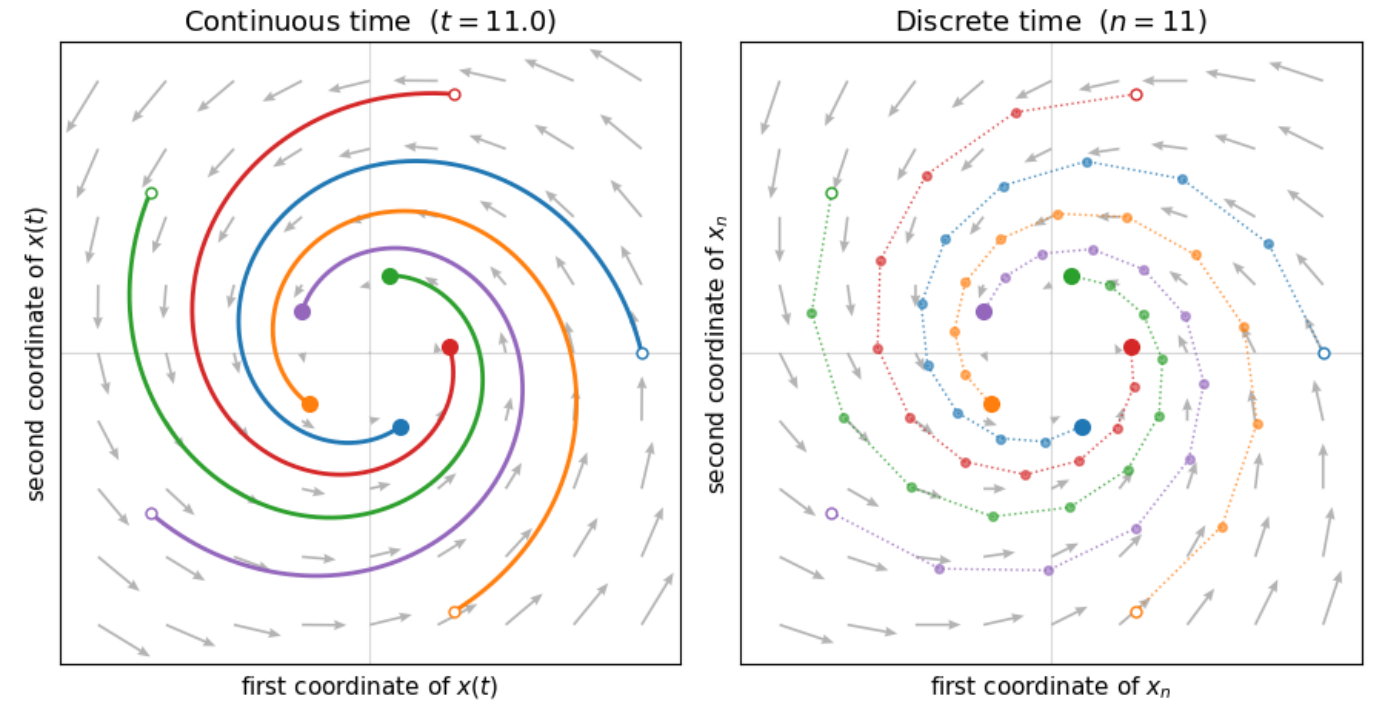


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

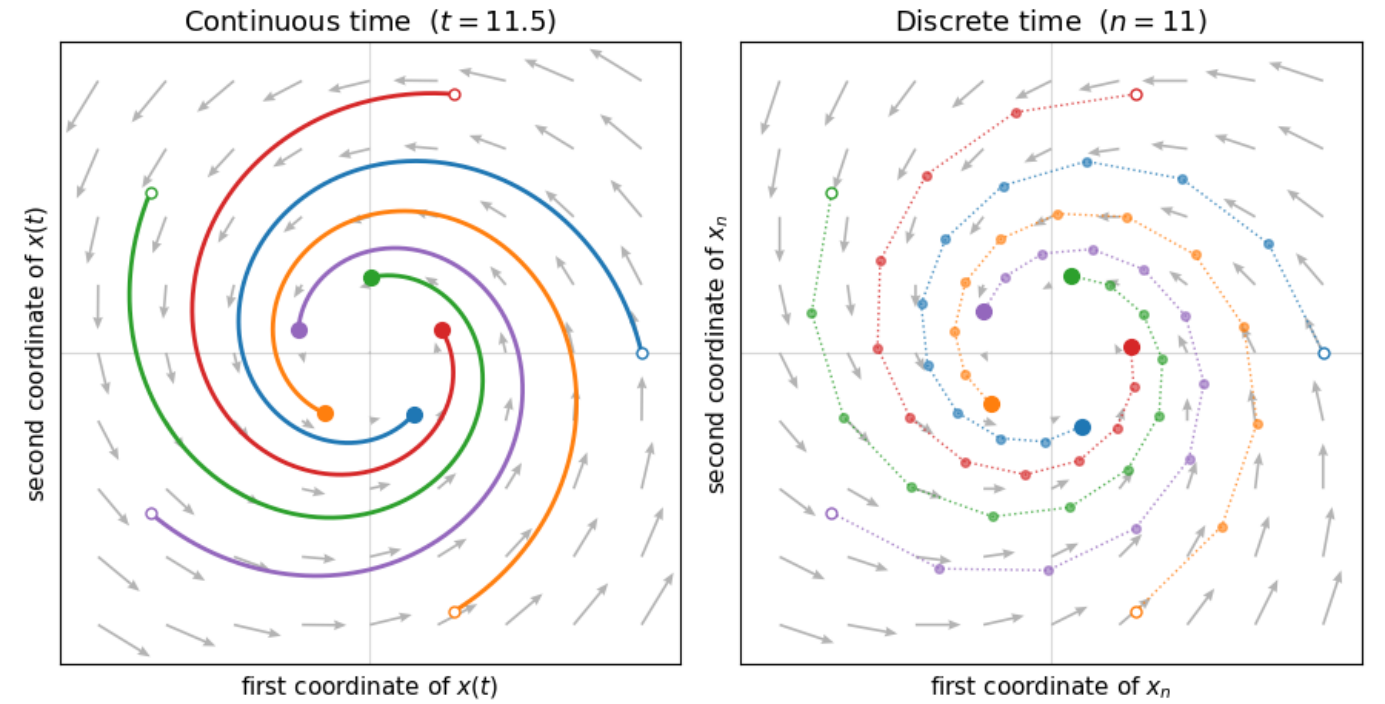


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.

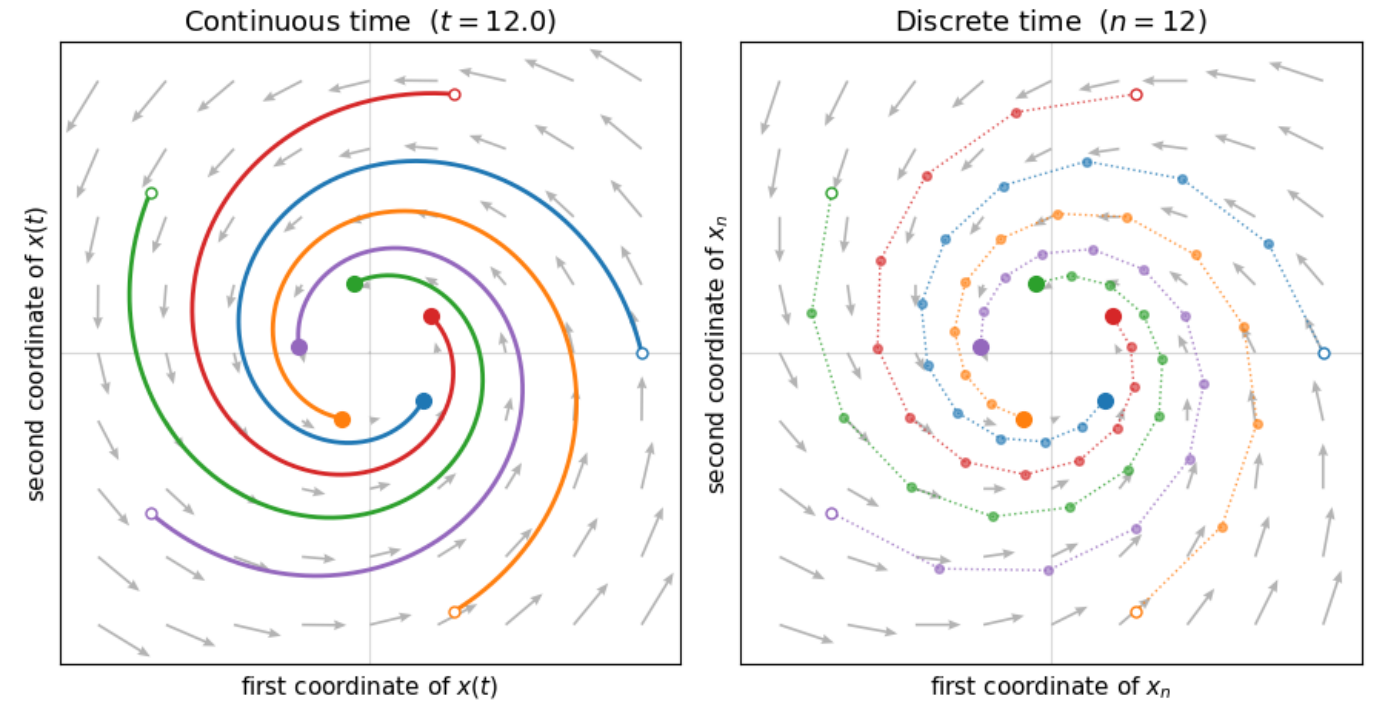


Fun Note: Recurrences are discrete-time differential equations

Continuous time (differential eq.)
$x'(t) = k\,x(t) \quad \Rightarrow \quad x(t) = x(0)\,e^{kt}$

Discrete time (recurrence)
$a_n = s\,a_{n-1} \quad \Rightarrow \quad a_n = a_0\,s^n$

Below, each coordinate of obeys $a_n = 1.6\,a_{n-1} - 0.8\,a_{n-2}$, which gives **spirals**.



Most important results from the textbook – Part 2

Thm 23.17 shows how to find a polynomial expression to generate the first few terms of a sequence.

Theorem 23.17 Let $a_0, a_1, a_2, \ldots$ be a sequence of numbers. The terms a_n can be expressed as polynomial expressions in n if and only if there is a nonnegative integer k such that for all $n \geq 0$ we have $\Delta^{k+1} a_n = 0$. In this case,

$$a_n = a_0 \binom{n}{0} + (\Delta a_0) \binom{n}{1} + (\Delta^2 a_0) \binom{n}{2} + \cdots + \left(\Delta^k a_0\right) \binom{n}{k}.$$

Group exercises